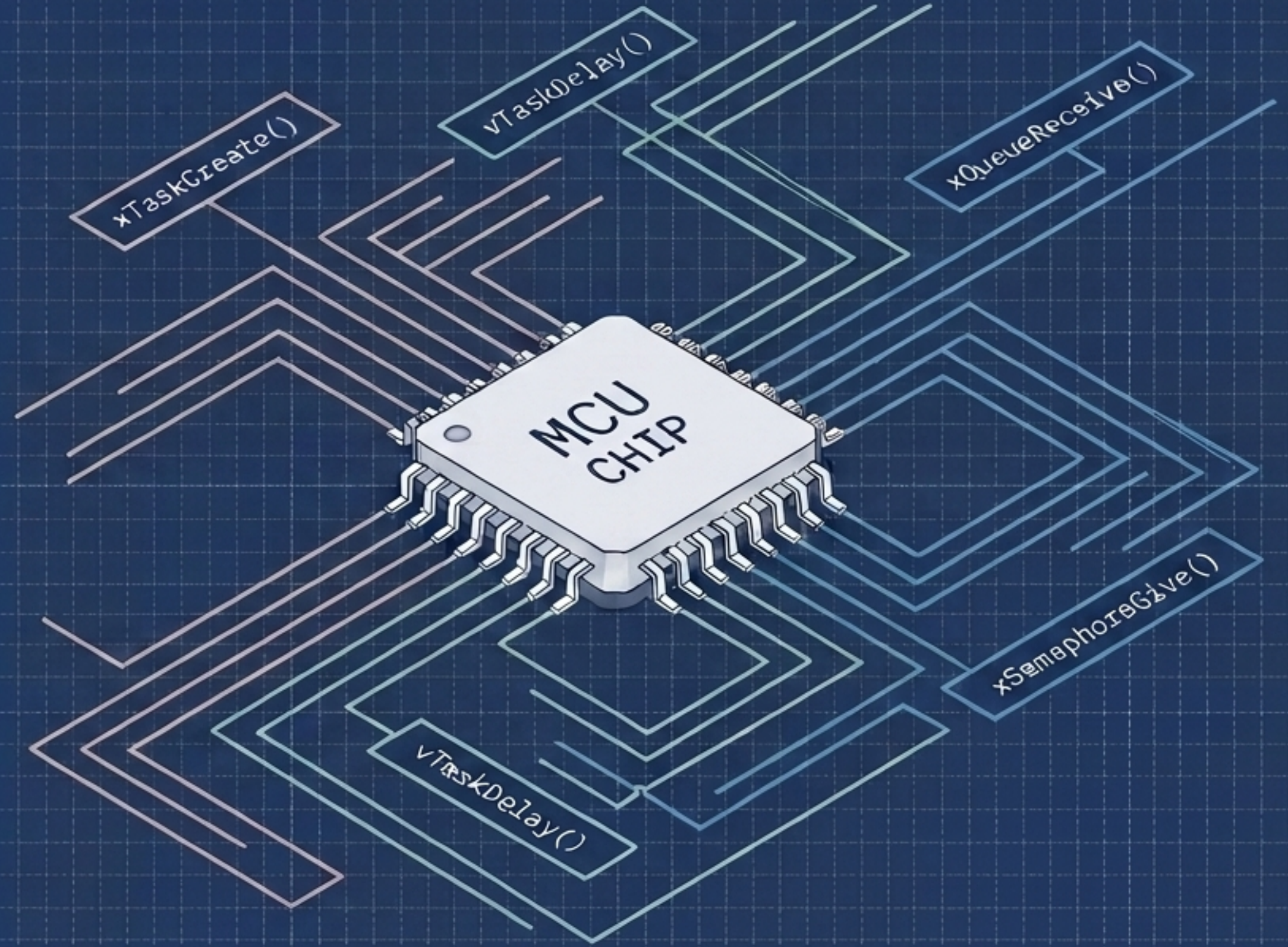
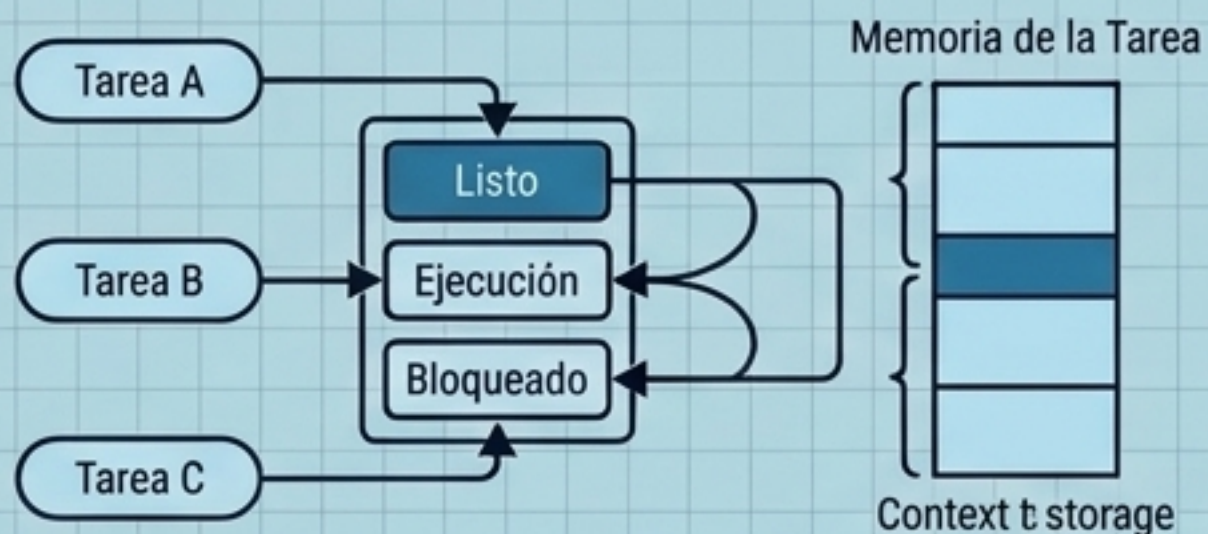


FreeRTOS sistema operativo en tiempo real



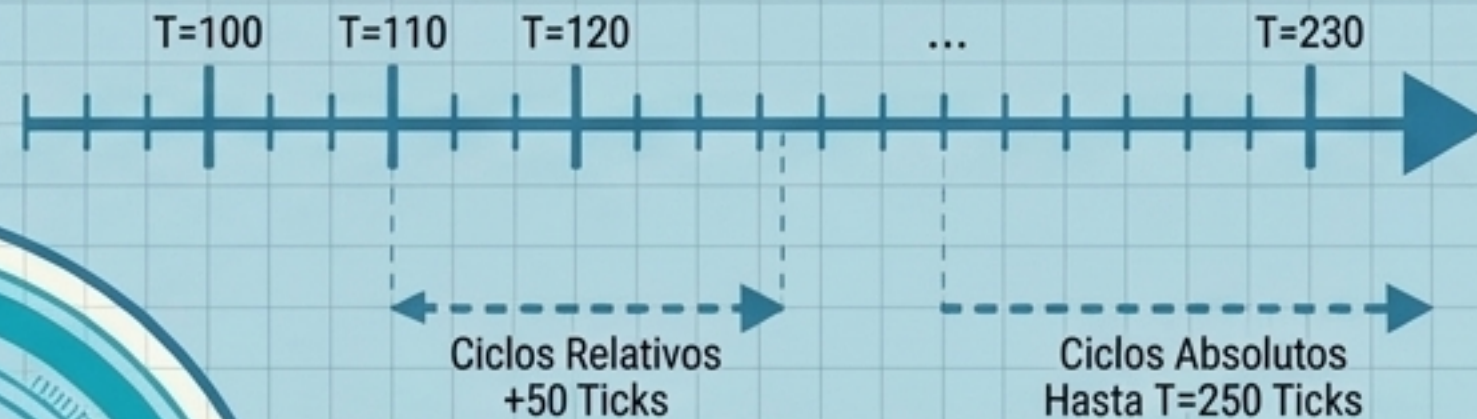
Unidades de Ejecución (Tareas)

Gestión de estados y memoria de la tarea.



Gestión del Tiempo (Ticks)

Ciclos relativos y absolutos.

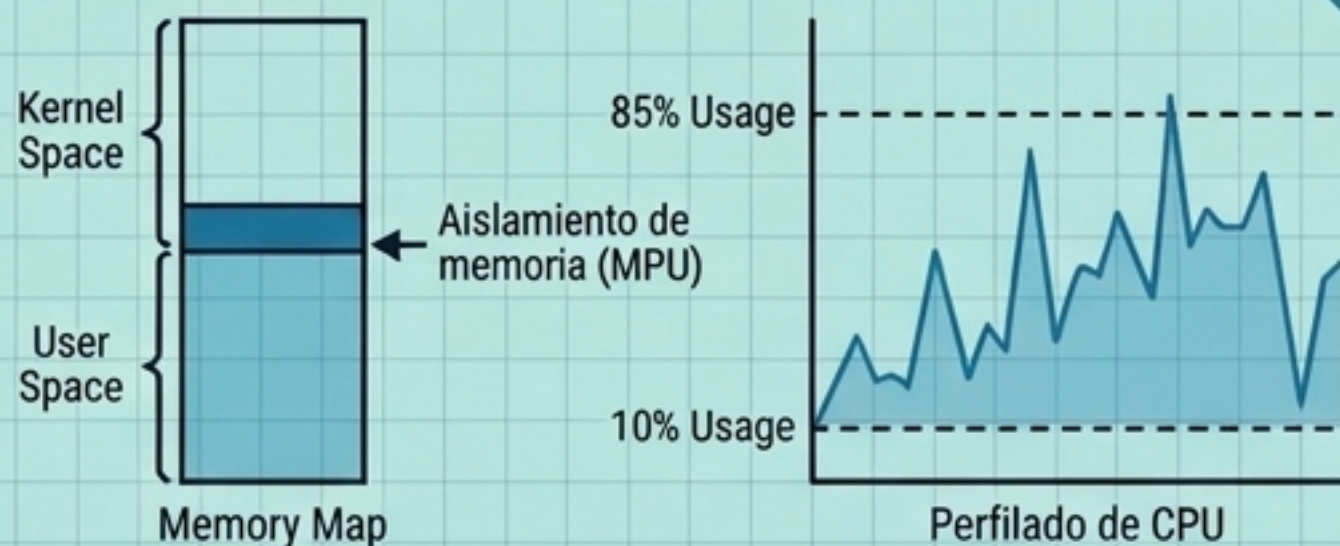


El Cerebro: Planificador

Ejecuta la regla de oro: La tarea Ready de mayor prioridad siempre obtiene la CPU.

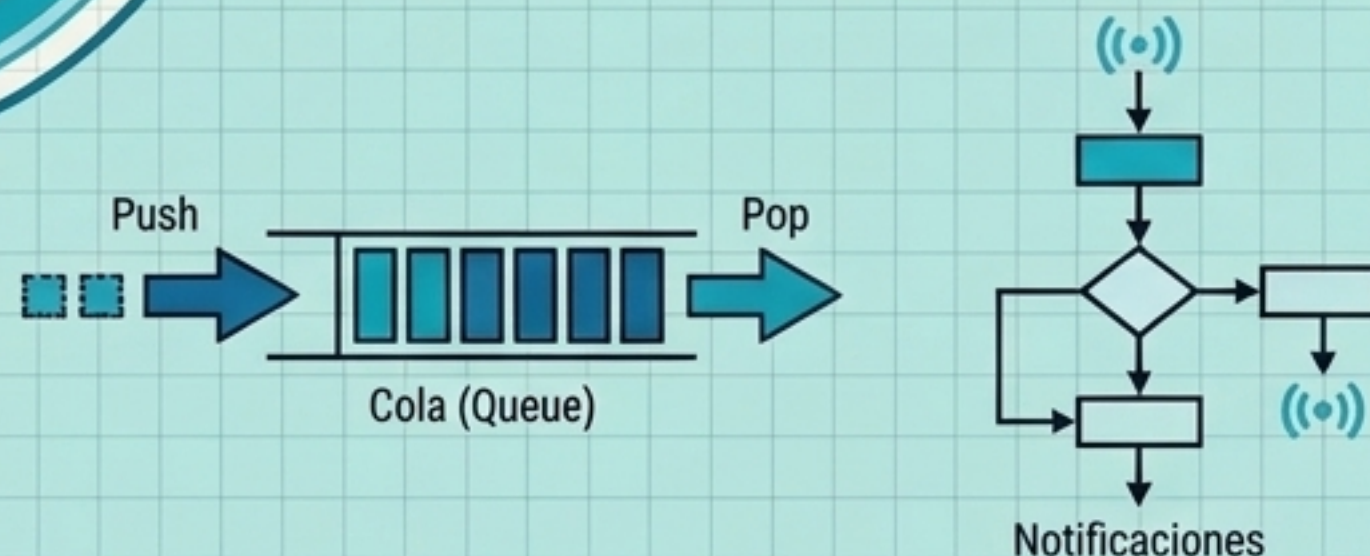
Seguridad y Diagnóstico

Aislamiento de memoria (MPU) y perfilado de CPU.

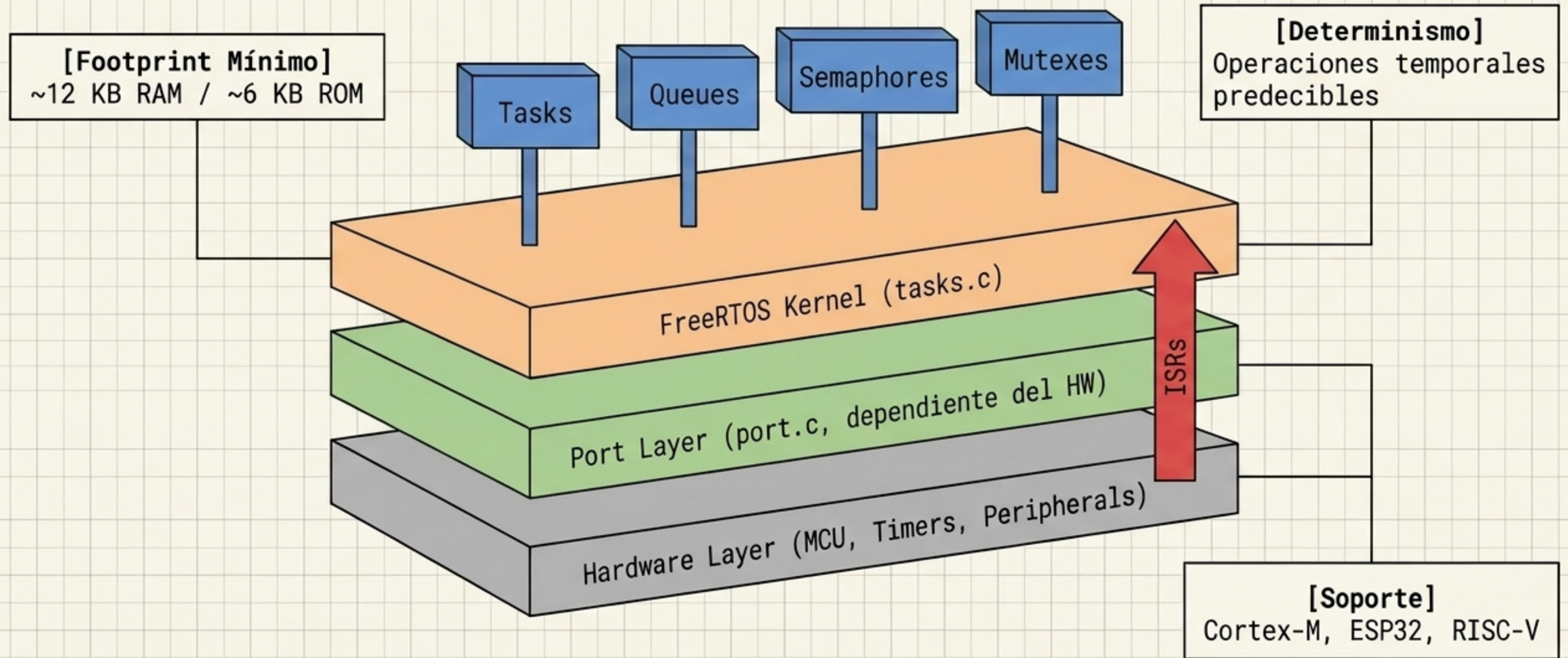


Canales de Comunicación

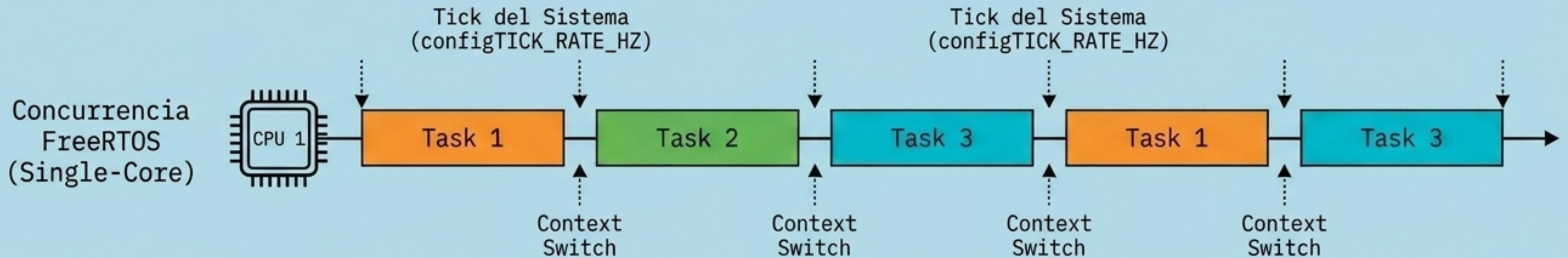
Colas (Queues) y Notificaciones para transferir datos y señales.



Arquitectura Interna de FreeRTOS



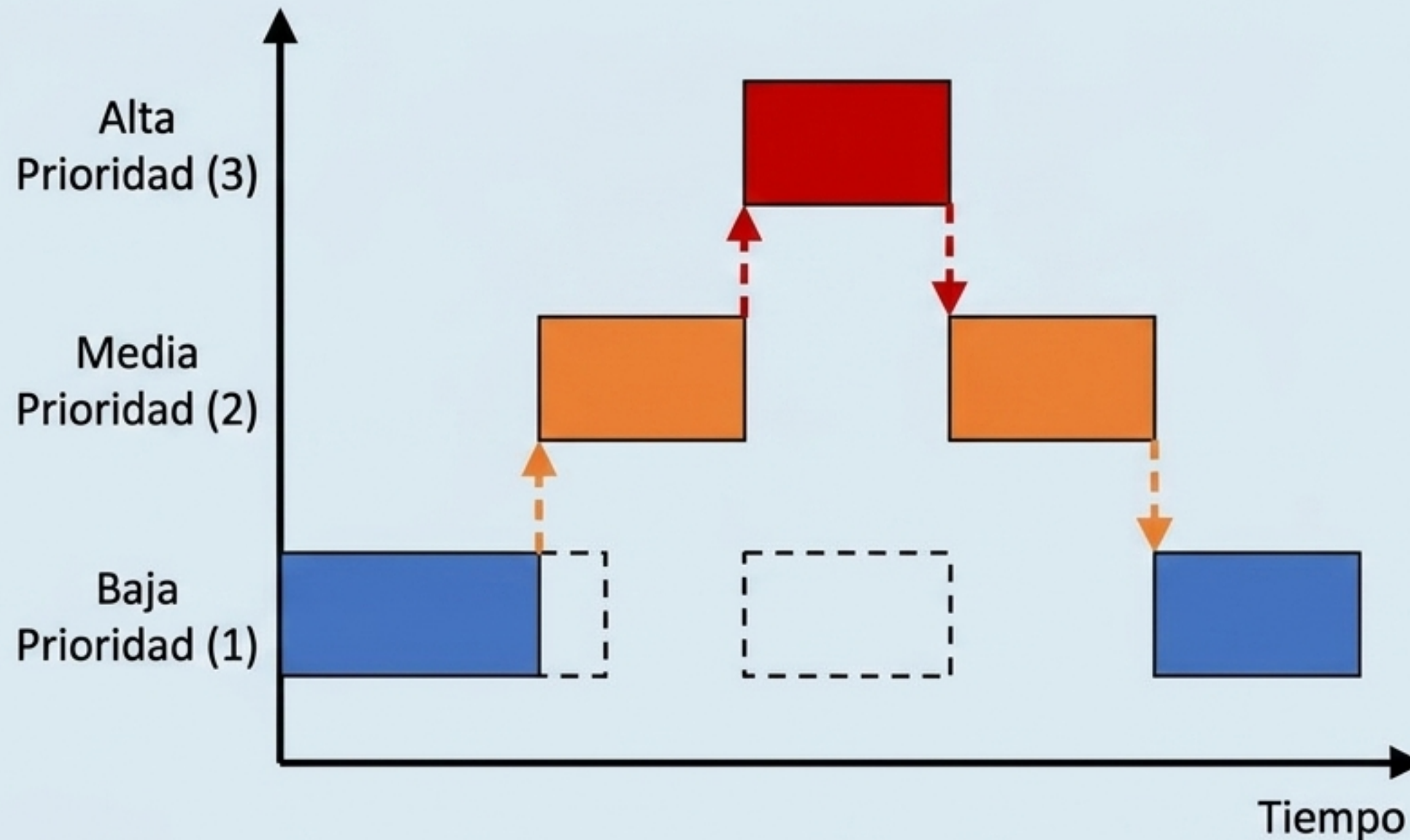
Concurrencia vs. Paralelismo



La concurrencia crea la ilusión de paralelismo mediante el Context Switching rápido en cada Tick del sistema.

El Dictador Benevolente: La Regla de Preemption

El planificador asegura que la CPU esté **siempre** en manos de la tarea Ready con el número de prioridad más alto.

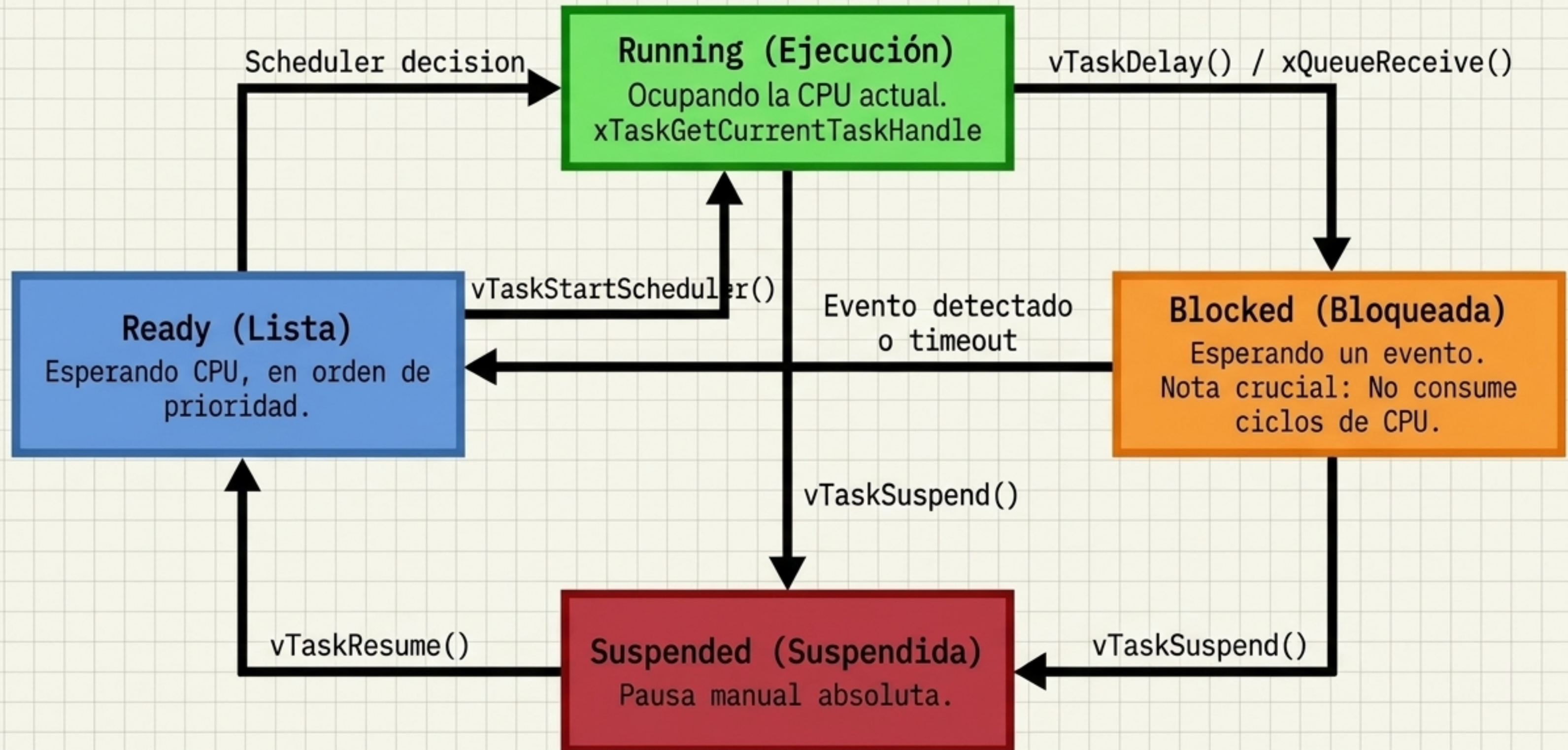


vTaskPrioritySet() / uxTaskPriorityGet(): Modificación de jerarquías en tiempo de ejecución.

vTaskSuspendAll(): Pausa los cambios de contexto sin deshabilitar las interrupciones por hardware.

xTaskResumeAll(): Restaura el planificador.

Máquina de Estados de la Tarea



Anatomía y Creación: xTaskCreate

Embedded Blueprint

```
xTaskCreate(  
  vMiTarea, // Función  
  'MiTarea', // Nombre  
  256, // Tamaño Stack  
  NULL, // Parámetro  
  3, // Prioridad  
  &xHandle // Handle  
);
```

RAM

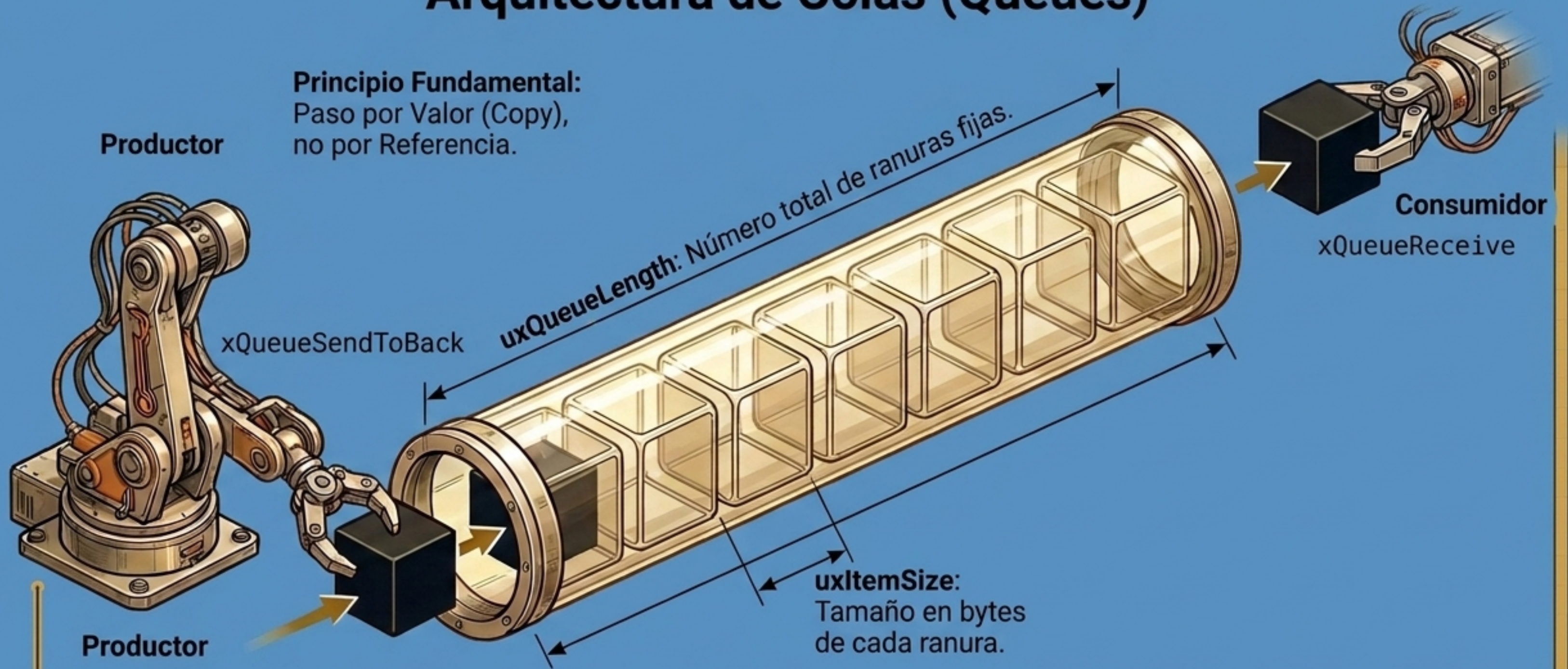
TCB (Task Control Block)
(Almacena el estado,
prioridad, nombre)

Stack Allocation
(Memoria dinámica
basada en el parámetro
'Tamaño Stack')

Task Handle
(Almacena la referencia
de la tarea)

ADVERTENCIA: Nunca usar return. Si la tarea termina su ciclo de vida lógico, debe llamar obligatoriamente a **vTaskDelete(NULL)**.

Arquitectura de Colas (Queues)



FreeRTOS copia físicamente los datos a la cola. Para grandes volúmenes de datos, es arquitectónicamente superior encolar punteros a los datos para ahorrar RAM y ciclos de CPU, asumiendo la responsabilidad de gestionar la memoria apuntada.

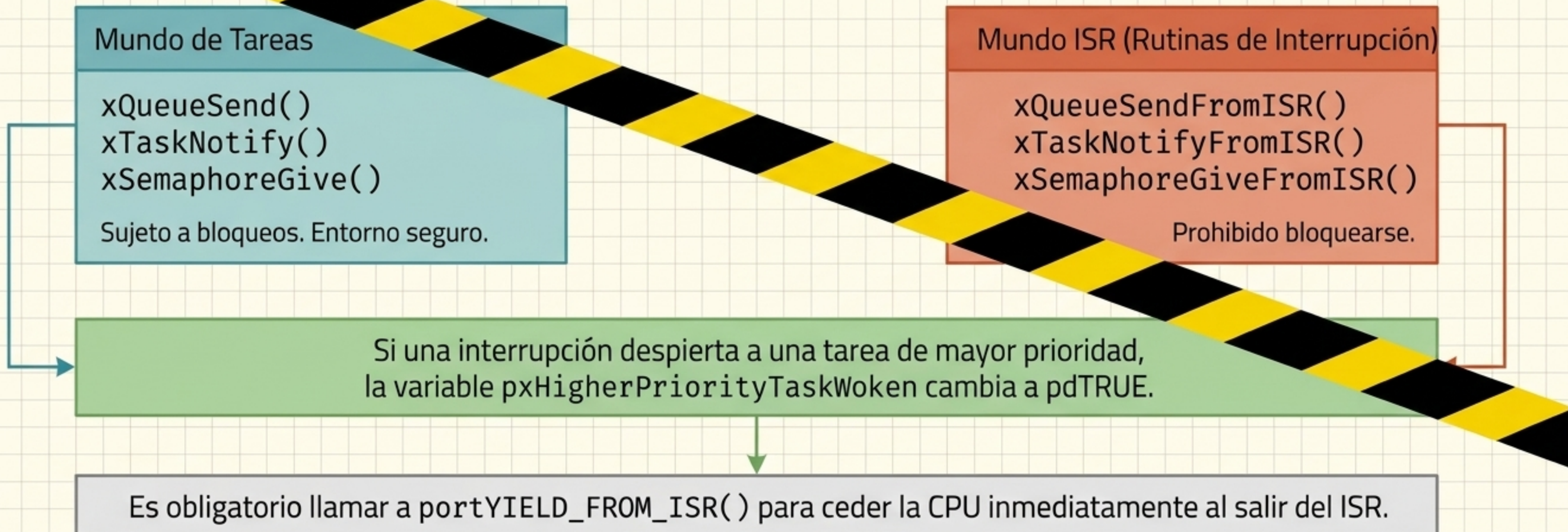
Matriz de Decisión: Eligiendo la Primitiva Correcta

Objetivo de Ingeniería	Herramienta Óptima	Costo RAM	Velocidad
Enviar datos estructurados entre tareas	Cola Estándar	Alto (Copias)	Moderada
Avisar a una tarea de un evento	Task Notification	Cero	Máxima
Proteger recursos compartidos	Mutex	Bajo	Lenta (Herencia)
Sincronización ISR -> Tarea	Semáforo Binario	Bajo	Alta

Arquitectura robusta no es usar cada función, es elegir la primitiva más ligera para el problema correcto

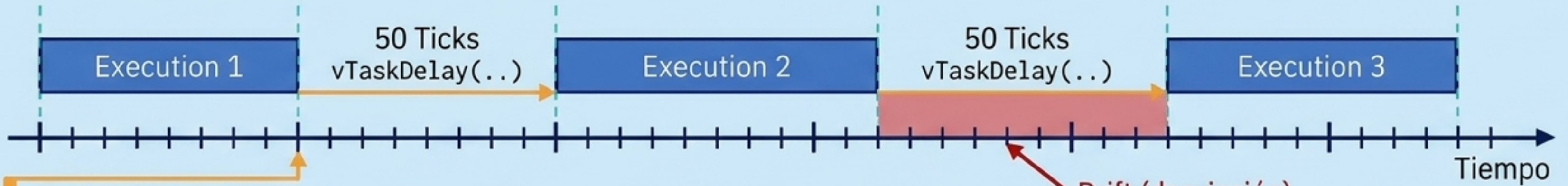
La Regla de Oro de las Interrupciones

¡Nunca llame a funciones **API estándar desde un ISR!**
Utilice únicamente las variantes diseñadas para interrupciones.



Demora Relativa

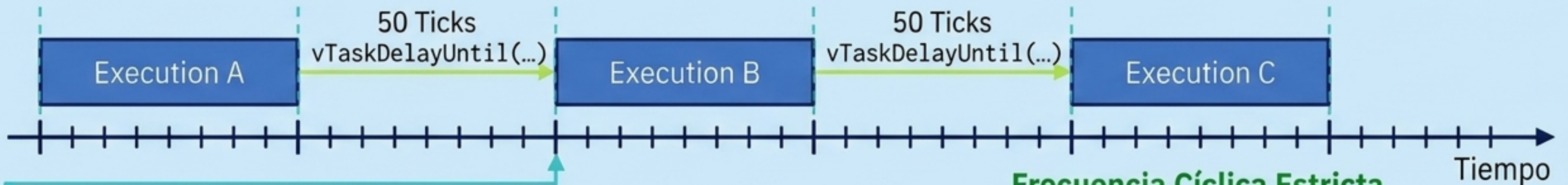
`vTaskDelay(pdMS_TO_TICKS(50))`



Bloquea la tarea por 50ms a partir de este exacto momento.
Riesgo de deriva de tiempo (drift) si la tarea es interrumpida.

Demora Absoluta

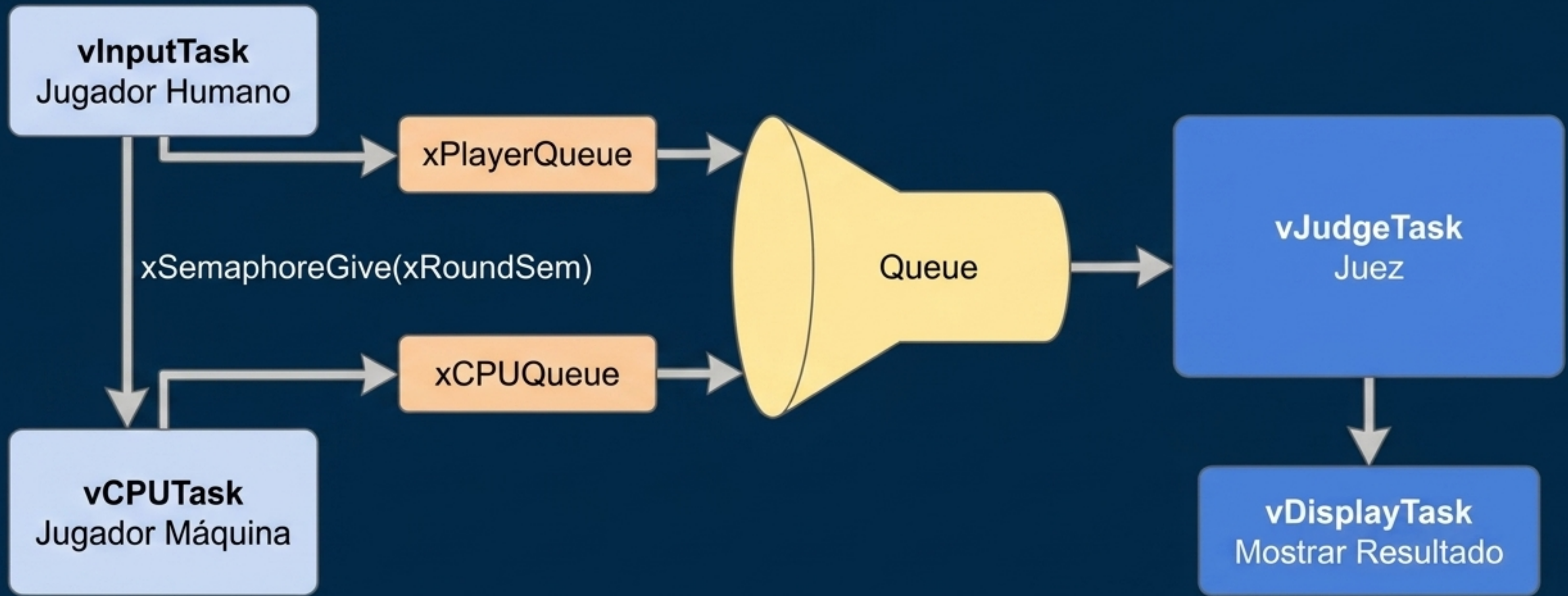
`vTaskDelayUntil(&xLastWakeTime, 50)`



Bloquea hasta alcanzar una marca de tiempo absoluta calculada.
Perfecto para frecuencias de ejecución cíclica estricta.

🕒 El latido del sistema se consulta con `xTaskGetTickCount()`, cuya resolución es definida en `configTICK_RATE_HZ`.

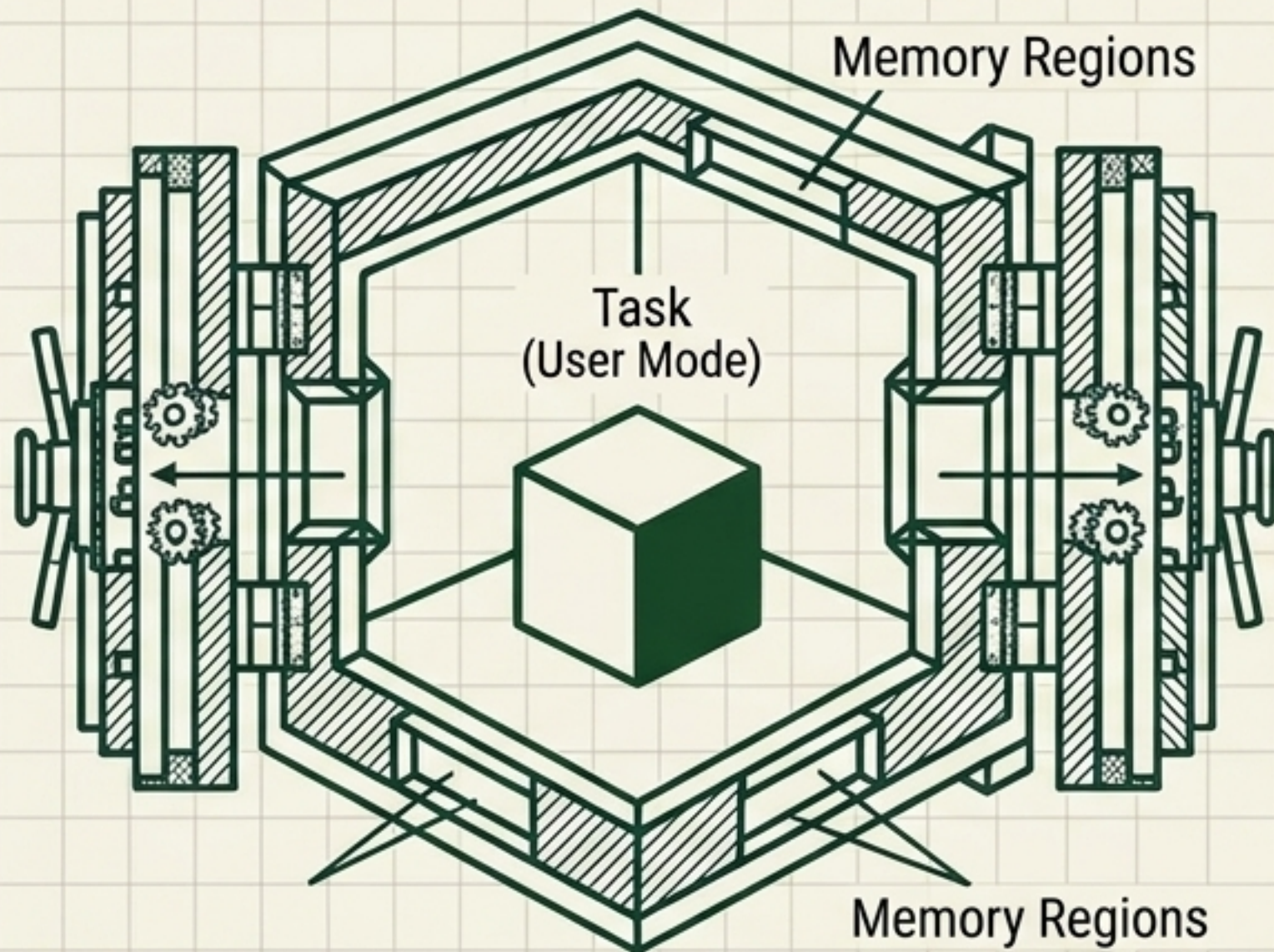
Flujo de Interaccion: Juego Piedra Papel Tijeras con FreeRTOS



sincronizacion dinamica de rondas con semaforo binario.

Seguridad y Perfilado de Memoria

Protección de Memoria



Aislamiento a nivel hardware. Protege al núcleo de bugs en el código de usuario.

Telemetría de Terminal

```
Name      State  Priority Stack  Num
*****
Math7      R      4        331   29
QConsB2    R      0         53   14
IDLE       R      0          6    0
```

Stack High Water Mark
(uxTaskGetStackHighWaterMark):
Muestra el espacio libre mínimo histórico. Si se acerca a 0 =
Riesgo inminente de desbordamiento.